LACORO
WAREHOUSE RL CHALLENGE 2025

Official Competition Manual

Simulation, Reinforcement Learning, and Social Navigation

LACORO

Latin American
Summer School
on Robotics

9 – 13 December 2025
Rancagua, Chile

Context:
Summer School LACORO 2025

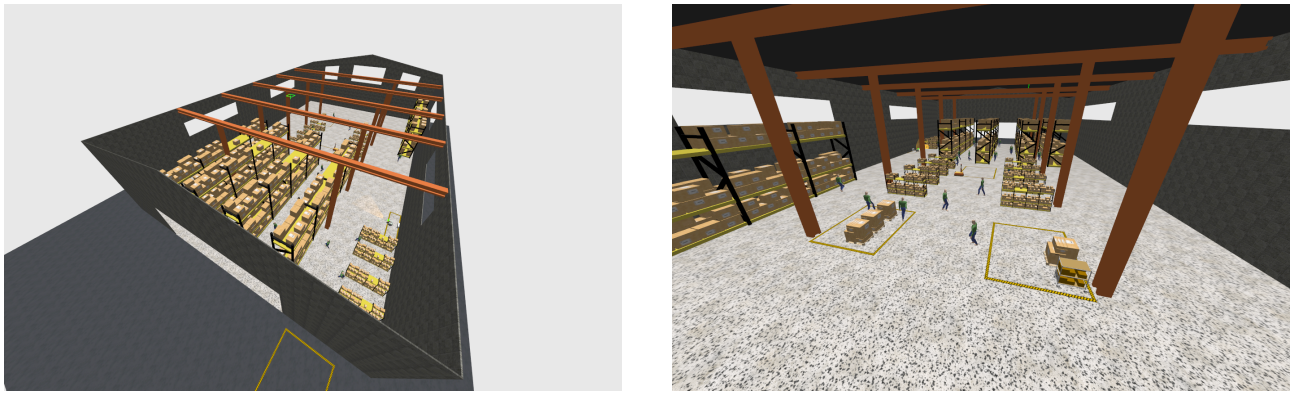
Technologies:
Reinforcement Learning · ROS 2 · Gazebo Fortress

Coordinators / Responsibles:
Simon Martinez R (simon.martinez@uantof.cl) · Stefan Escaida (stefan.escaida@uoh.cl)

Contents

1 Competition Context

The **LACORO Warehouse RL Challenge 2025** is a simulation-based competition focused on developing Reinforcement Learning (RL) agents capable of safely and efficiently navigating in a warehouse shared with simulated human workers (actors), see Figure 1. The simulation is implemented in **Gazebo Fortress** and is integrated with **ROS 2** to manage robot control and sensor data.



(a) Outside view of the warehouse scenario.

(b) Inside view with shelves, actors, and robot.

Figure 1: Warehouse simulation framework in Gazebo Fortress for the LACORO Warehouse RL Challenge 2025.

1.1 Main Objective

The main objective of the challenge is to **design and train a control policy** that enables a mobile robot to navigate between different points in the warehouse:

1. Without colliding with workers, shelves, or obstacles.
2. Respecting internal “traffic rules” inside the warehouse.
3. Completing a transport mission by visiting a sequence of stations (pickup and delivery points).

In practical terms, each team must develop an autonomous navigation policy based on Reinforcement Learning (RL). This policy must process the robot’s sensor data and produce safe velocity commands that allow the robot to navigate, avoid actors, and complete pickup–delivery missions (that means visit start and final points) inside the warehouse.

1.2 Motivation

The **LACORO Warehouse RL Challenge** addresses the growing need for intelligent logistics and safe human–robot interaction in Industry 4.0 environments. Participants are required to train a mobile robot using RL to:

- Safely navigate in a dynamic environment with human actors.
- Detect and avoid static and dynamic obstacles.
- Make real-time decisions that prioritize safety and mission success.

The solutions developed in this challenge are directly transferable to real-world applications such as service robotics, industrial inspection, and autonomous logistics systems, where the ability of an AI agent to move safely and reliably in complex environments is essential.

1.3 Prizes and Awards

The winning team of the **LACORO Warehouse RL Challenge 2025** will receive a cash prize of **200 USD**, in addition to official recognition during the LACORO Summer School closing session.

2 Target Audience and Teams

- The competition is open to Undergraduate, Master's, and PhD students.
- Teams may consist of up to **4 members**.

It is recommended (but not strictly required) that team members have prior knowledge in:

- ROS 2 and Gazebo simulation.
- Reinforcement Learning fundamentals.
- Perception and sensor data processing.
- Software engineering and integration of robotics systems.

3 Installation and System Requirements

This section provides a high-level description of the software and hardware requirements to run the LACORO Warehouse RL Challenge simulation and RL environment.

3.1 Recommended Software Environment

- **Operating System:** Ubuntu 22.04 LTS (64-bit).
- **ROS 2 Distribution:** ROS 2 Humble or compatible version.
- **Gazebo:** Gazebo Fortress.
- **Python:** Python 3.8+.
- **RL Libraries (examples):** `stable-baselines3`, `gymnasium`, `numpy`, `tensorboard`, and optionally deep-learning frameworks such as `PyTorch` or `TensorFlow`.

The libraries and installation dependencies will be provided through a **script** and documented in the repository.

The final installation must properly integrate the different modules and packages to ensure a correct workflow. A recommended installation pipeline is presented in Fig. 2.

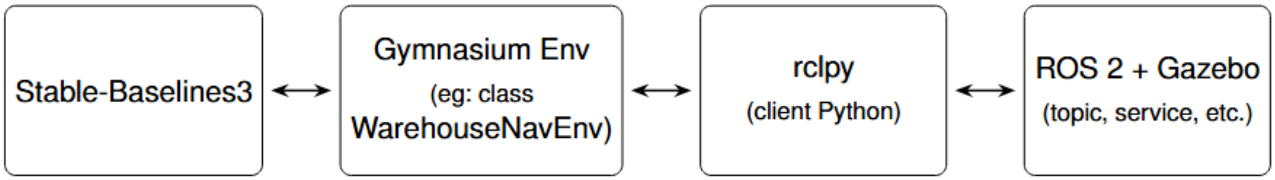


Figure 2: Example of the pipeline diagram of the system to be implemented for the challenge

3.2 Hardware Requirements

- Quad-core CPU or higher (recommended).
- 16 GB RAM or more for comfortable parallel simulation.
- Dedicated GPU (NVIDIA) recommended for training deep RL models.

3.3 Repository and Workspace Setup

The official challenge code is distributed via a public repository in GitHub (`https://github.com/roboticsuantof/warehouse_simulator.git`). A typical ROS 2 workspace structure is:

Listing 1: Example ROS 2 workspace layout

```
~/lacoro_ws/
  src/
   warehouse_simulator/
   warehouse_rl/ # e.g. here could be develop RL package for training
  ...
```

Recommended build steps:

Listing 2: Building the workspace

```
cd ~/lacoro_ws
colcon build
source install/setup.bash
```

Comprehensive information on the installation process and workspace building instructions is available in the README file of the `warehouse_simulator` repository.

4 Quick Start Guide

This section provides a minimal set of steps to help participants quickly verify that the simulator and environment are correctly installed.

4.1 Step 1: Launch the Warehouse Simulation

After building and sourcing the workspace:

Listing 3: Launching the main warehouse simulation

```
ros2 launch warehouse_simulator warehouse_simulation.launch.py world_name:=
   warehouse_full gui:=true
```

You should see:

- The warehouse world in Gazebo Fortress.
- The service mobile robot spawned in the environment.

- In the full scenario (`warehouse_full.sdf`), human actors move along their trajectories.

To load one of the training worlds, replace `warehouse_full` with `warehouse_01`, `warehouse_02`, or `warehouse_03`.

4.2 Step 2: Verify Robot Topics

Open a new terminal and source the workspace:

```
source ~/lacoro_ws/install/setup.bash
ros2 topic list
```

You should find (names may vary slightly):

- `/robot/cmd_vel` – velocity commands.
- `/robot/lidar2d/scan` – 2D LiDAR scan.
- `/robot/lidar3d/points` – 3D LiDAR point cloud.
- `/robot/imu` – IMU data.
- `/robot/camera/rgb/image` – RGB camera image.
- `/robot/camera/depth/image` – depth image.

4.3 Step 3: Send a Simple Velocity Command

As a quick test, send a constant velocity command:

Listing 4: Publishing a test velocity command

```
ros2 topic pub /robot/cmd_vel geometry_msgs/Twist "{linear: {x: 0.6}, angular: {
    z: 0.3}}"
```

If everything is working, the robot should move in the simulation.

4.4 Step 4: RL Python Environment

Instructions for properly setting up the Python environment for the challenge are available in the README file of the `warehouse_simulator` repository. The complete environment can be generated using the accompanying **script**. Teams are then free to design and implement their own RL training pipelines using this environment.

5 Schedule (3 Afternoons)

The event is designed to take place on Wednesday, Thursday, and Friday during the LACORO Summer School 2025. Optionally, results are formalized on Saturday.

5.1 Day 1 (Wednesday) – Kickoff & Simulation Setup

- Presentation of the challenge, rules, and evaluation metrics.
- Explanation of the simulator, robot, and sensors.
- Delivery and testing of the **Starter Kit** (ROS 2 environment + Gazebo Fortress packages).
- **Hands-on Setup:** Each team must:

  - Build the workspace.
  - Launch the warehouse simulation.
  - Demonstrate that the robot moves in the warehouse without errors.
  - Complete a simple task (e.g., reaching a single static goal without collision).

5.2 Day 2 (Thursday) – Development

- **Development:** Teams design and train their own RL approach (algorithm, network architecture, reward function, exploration strategy, etc.). The three training scenarios can be used to:
  - Modify shelf layouts.
  - Adjust actor patterns (where applicable).
  - Vary random seeds for robustness.
- **Daily Deliverable:**
  - A frozen version of the policy (`policy_day2_checkpoint.zip` or similar).
  - A brief report (e.g., `.json`, `.yaml`, `.csv`, or `.txt`) summarizing key metrics and including at least one screenshot:
    * Goals reached: e.g., 2/3.
    * Static collisions: e.g., 0.
    * Actor collisions: e.g., 1.
    * Time: e.g., 142 s.
    * Distance: e.g., 35.7 m.

5.3 Day 3 (Friday) – Finals & Presentations

- **Code Freeze (18:00 hrs):** After this time, solutions cannot be modified.
- **Official Evaluation:** The final model is evaluated in the `warehouse_full.sdf` scenario, which includes human actors. Evaluation consists of:
  - Episodes with different seeds.
  - Different initial and goal positions selected from predefined sets.
  - Random actor motion patterns.
  - Variations in perception noise.
- **Team Presentation (5 minutes):**
  - RL approach used (algorithm, architecture, etc.).
  - Reward design and shaping.
  - Techniques for generalization and robustness.
  - Lessons learned and limitations.

5.4 Day 4 (Saturday) – Results

- Formal publication of the final results and the announcement of the winning teams.

6 Task Scenario

This section describes the simulated warehouse environment, the mobile robot, and the mission that defines the navigation task for the challenge. All teams will train their models in the provided scenarios and will be evaluated in the most complex one, where human actors are present and multiple start–goal configurations are tested.

The challenge provides four simulation scenarios: three training worlds (`warehouse_01.sdf`, `warehouse_02.sdf`, `warehouse_03.sdf`) and one final evaluation world (`warehouse_full.sdf`). Teams are expected to train and tune their RL models using the training scenarios. The official evaluation is performed only in `warehouse_full.sdf`, which includes human actors and multiple predefined start–goal configurations. These evaluation configurations are not modified by the teams and are used exclusively by the organizers during final scoring.

6.1 Environment: Warehouse with Workers

The environment is a simulated industrial warehouse in Gazebo Fortress, as shown Figure 3, featuring:

- Aisles and shelves.
- Picking and delivery zones.
- Loading and unloading areas.
- Static obstacles such as pallets, boxes, and walls.
- In the evaluation scenario (`warehouse_full.sdf`), human actors walking along predefined or randomized trajectories.

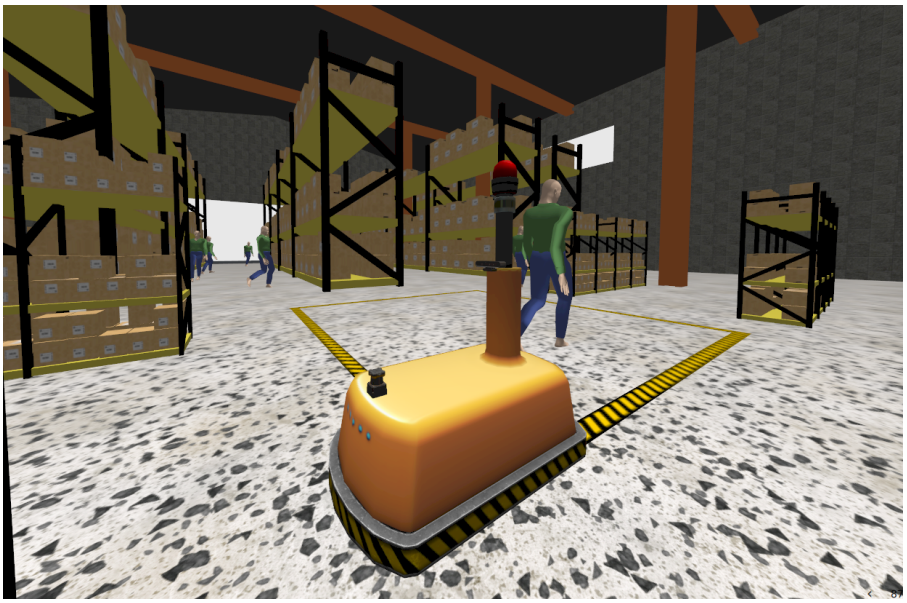


Figure 3: Simulated warehouse with aisles, shelves, and key pickup/delivery zones.

6.2 Robot and Sensors

The robot is a differential-drive service mobile robot. Its control input is a pair of linear and angular velocity commands (v, ω), subject to the following bounds:

- Maximum linear speed: $v_{\max} \approx 0.6$ m/s.
- Maximum angular speed: $\omega_{\max} \approx 1.0$ rad/s.

The robot is equipped, as shown Figure 4, with:

- **2D LiDAR** (360° horizontal scan). Topic: `/robot/lidar2d/scan`.
- **3D LiDAR** (partial or full volumetric scan). Topic: `/robot/lidar3d/points`.
- **IMU** (accelerometers + gyroscope). Topic: `/robot/imu`.
- **RGB-D camera** facing forward. Example topics:
  - `/robot/camera/rgb/image`
  - `/robot/camera/depth/image`

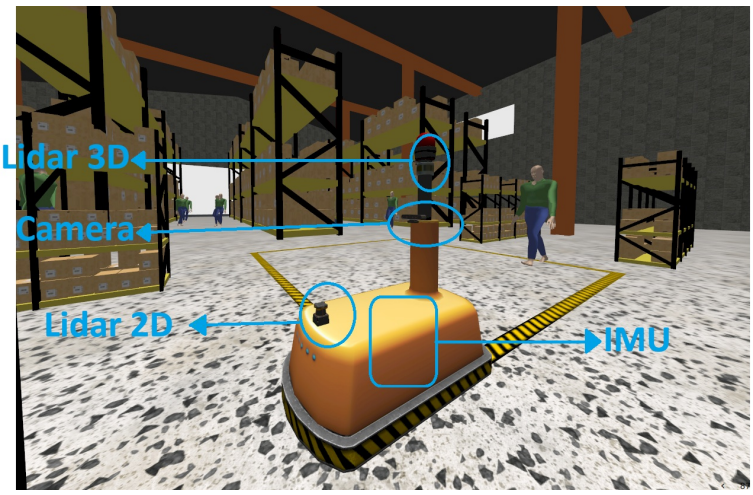


Figure 4: Sensor configuration of the mobile robot used in the challenge.

6.3 Mission Definition

- Each episode lasts $T = 300$ s (5 minutes) of simulation time.
- The mission consists of N pickup–delivery pairs. For each pair $i \in \{1, \ldots, N\}$:
  - The pickup–delivery pairs are defined in the file `config/mission_goals.yaml`, which is provided to the teams. The same file is used in the evaluation world, ensuring consistency between training and final testing.
  - The robot must move from the pickup location to the corresponding delivery location.
- The main objective is to:
  - Visit as many pickup–delivery pairs as possible.
  - Avoid collisions with static obstacles and actors.
  - Minimize mission time and traveled distance.

7 RL Problem Specification

This section formalizes the navigation problem as a Reinforcement Learning task. It describes the observation and action spaces available to the agent, the episode termination conditions, and general guidelines for reward design. The specific RL algorithm, architecture, and training strategy are entirely defined by each team, as long as the resulting policy can safely navigate the warehouse and respect the competition rules.

The challenge does not impose any specific RL algorithm or architecture. Teams are free to use any RL method, deep-learning framework, or hybrid approach, as long as the final behavior is fully autonomous and compatible with the ROS2/Gazebo evaluation pipeline.

7.1 Observation Space (State)

The following elements can be included in the observation vector of the RL agent. Teams must include **at least one** of these sources:

- Processed 2D LiDAR distance vector (e.g., 180 rays).
- Subsampled 3D LiDAR information (e.g., 2D projection or min/max height per sector).
- Estimated robot pose (e.g., (x, y, θ) in a global frame or relative to the current goal).
- Current robot velocity (v, ω).
- Goal information (relative position (x, y) and goal index or ID).
- *Optional:* Partial information about the k nearest actors (e.g., relative positions or distances).

7.2 Action Space

- **Continuous actions:** (v, ω) directly mapped to the velocity commands of the robot, respecting the physical limits.
- **Alternative (Discrete):** A finite set of actions such as:
  - Forward.
  - Forward + soft left turn.
  - Forward + soft right turn.
  - Hard left turn.
  - Hard right turn.
  - Stop.

7.3 Episode Termination Conditions

An episode terminates when **any** of the following occurs:

- The robot completes the predefined number of deliveries.
- A collision with a human actor occurs (episode is invalid for scoring).
- Optional: The robot is detected as “stuck” (no progress after a certain time).

7.4 Reward Design (Guidelines)

The challenge does not impose a fixed reward function, but typical components may include:

- Positive reward for reaching pickup or delivery points.
- Negative reward for collisions (especially with actors).
- Penalties for large deviations from the shortest path.
- Time penalty to encourage efficient trajectories.
- Optional: Safety margin rewards for maintaining a safe distance to actors.

8 Resources Provided to Teams

The organization provides the following resources, which must be used for the challenge.

8.1 Simulator and Worlds

A ROS 2 package `warehouse_simulator` containing:

- **Training Worlds:** Three scenarios (`warehouse_01.sdf`, `warehouse_02.sdf`, `warehouse_03.sdf`) with increasing layout complexity and difficulty, designed primarily for training and debugging.
- **Evaluation World:** The official evaluation scenario, `warehouse_full.sdf`, which includes human actors and the full mission configuration.
- **Robot Model:** A service mobile robot available as `warehouse_robot`.
- **Human Actors:** Simulated workers with predefined or randomized walking behaviors in the evaluation world.

8.2 Starter Kit

The Starter Kit includes all the necessary tools to begin training immediately:

- A setup script that installs the Python virtual environment, RL libraries, and required dependencies.
- Environment variables and ROS 2 setup instructions for ensuring that Python scripts can communicate with the Gazebo simulation.
- Example scripts demonstrating:
  - How to reset the environment from Python.
  - How to read observations (LiDAR, IMU, camera, goal position).
  - How to publish velocity commands.
- A minimal working baseline (optional) illustrating a simple rule-based navigator, provided solely for testing the communication pipeline.

These resources are meant to ensure that all teams can start from a functional environment and focus on designing and training their own RL models.

8.3 Evaluation Infrastructure

A jury will fairly evaluate the system composed of ROS 2, Gazebo, and the RL model, executed under various random seeds and scenario configurations, generating standardized metric files for comparison.

The system consists of:

- **ROS 2 Evaluation Launcher:** A standardized launch file that loads the `warehouse_full.sdf` world, initializes robot and goals based on a deterministic seed, and executes the team's policy script.
- **Multi-Seed Evaluation Script:** A Python tool (`run_evaluation.py`) that:
  - Runs multiple episodes in `warehouse_full.sdf` with different seeds.
  - Randomizes actor motion patterns, initial robot poses, sensor noise, and goal assignments (from the predefined sets).
- **Metrics Logger:** For each evaluation, it records:
  - Goals reached.
  - Collisions (actors and static obstacles).
  - Mission time.
  - Traveled distance.
  - Optional: statistics on minimum distance to actors and obstacles.

  All results are exported to machine-readable formats (e.g., `.json`, `.csv`), which are then used to compute the official score.

9 Competition Evaluation

The LACORO Warehouse RL Challenge follows a **single competition modality** in which teams are free to design their own RL training pipelines and, if desired, combine them with other decision-making methods, as long as the final control of the robot is fully autonomous and respects the rules defined in this manual. This section specifies how solutions must be deployed, which deliverables are required, and how computational resources will be managed to ensure fairness across all participants.

9.1 Main Interest

During official evaluations:

- Each Team must develop it own RL model for safety navigation in the warehouse environment.
- The robot must operate **fully autonomously**.

- No teleoperation or human intervention is allowed during an episode.
- The policy cannot be changed during a batch of evaluation episodes.

9.2 Code and Deliverables

Each team must submit:

- Source code or an executable container (with clear instructions).
- Main execution script (e.g., `run_policy.py`) with a standardized interface.
- A short technical report describing:
  - RL algorithm and architecture (and any additional planning modules, if used).
  - Reward design.
  - Training procedure.
  - Key experimental results.

9.3 Computational Resources

- **Training:** Teams may use their own hardware (laptops).
- **Evaluation:** All final policies are evaluated on the same hardware managed by the organization to guarantee fairness.

10 Evaluation and Scoring System

The global evaluation metric combines three aspects:

- **Safety** (primary).
- **Mission Efficiency** (goals reached and time).
- **Navigation Quality** (path length).

10.1 Basic Metrics per Episode

For each episode, the following metrics are computed:

- **Goals reached (G):** $G \in \{0, 1, \ldots, N_{\max}\}$.
- **Collisions:**
  - C_{actor}: number of collisions with actors.
  - C_{static}: number of collisions with static obstacles.
- **Mission time (T):** total elapsed time until episode termination.
- **Travel distance (D):** total path length.
- **Optional Safety Distances:**
  - δ_o: minimum distance to static obstacles.
  - δ_a: minimum distance to actors.

10.2 Safety Rule

- Any collision with an actor $\Rightarrow$ **evaluation invalidated**, i.e., score $= 0$.
- Collisions with static obstacles incur a strong penalty.

10.3 Score per Evaluation (S_{ev})

The base score for an evaluation is defined as:

$$S_{\text{ev}} = \begin{cases} 0, & \text{if } C_{\text{actor}} > 0, \\ \alpha \cdot G - \beta \cdot C_{\text{static}} - \gamma \cdot \dfrac{T}{T_{\max}} - \delta \cdot \dfrac{D}{D_{\max}}, & \text{otherwise,} \end{cases}$$

where:

- α: weight for completed goals (e.g., 10–20 points).
- β: penalty per static collision.
- γ: penalty based on normalized mission time.
- δ: penalty based on normalized total distance.

Tie-breaking criteria may consider, in order:

1. Higher average number of goals reached.
2. Lower average number of collisions (both static and actors).
3. Lower average mission time.

The final evaluation set consists of a predefined collection of start–goal pairs inside `warehouse_full.sdf`. The exact pairs used for scoring are fixed by the organizers and are the same for all teams. Each seed samples a different subset of these configurations to measure the robustness of the trained policy.

10.4 Deliverables

On the final day of the challenge (Day 3), competitors must submit to the organizers a **presentation file (PPT)** describing the method developed, which will be explained during a **5-minute oral presentation**. This presentation will be evaluated by the judges. The PPT must include:

- The RL approach implemented (algorithm, architecture, etc.).
- Reward design and shaping strategy.
- Techniques used to improve generalization and robustness.
- Lessons learned and identified limitations.

10.5 Global Event Score

The policy is evaluated in the official scenario `warehouse_full.sdf` for K seeds. For each seed, multiple start–goal configurations may be tested. The final score is obtained as:

- The mean of S_{ev} over all evaluation episodes, and/or
- A lexicographic ranking based on (Goals, Safety, Time).

11 Frequently Asked Questions (FAQ)

Q1: Can we change the robot model?

No. The robot model provided in the `warehouse_simulator` package must be used to ensure a fair comparison.

Q2: Can we use external RL libraries or frameworks?

Yes. Any RL library or framework is allowed, as long as the final solution can be executed in the provided ROS 2/Gazebo environment.

Q3: What happens if the simulation crashes during evaluation?

If a crash is clearly attributed to simulation or infrastructure issues, the organization will repeat the affected episodes. If crashes are caused by the team's code, the episode may be invalidated.

Q4: Are we allowed to train in the official evaluation world?

Yes, but generalization is strongly encouraged. During official evaluation, multiple seeds and start–goal configurations are used in `warehouse_full.sdf`, so overfitting to a single configuration is discouraged.

12 Glossary

- **RL (Reinforcement Learning):** A learning framework where an agent learns by interacting with an environment and receiving rewards.
- **ROS 2:** Robot Operating System, version 2, a middleware for robotics.
- **Gazebo Fortress:** A 3D robotics simulator used for realistic physics and sensor simulation.
- **LiDAR:** Light Detection and Ranging sensor used to obtain distance information.
- **Policy:** The decision-making function of an RL agent that maps states to actions.

13 Contact and Support

For questions related to the challenge, please contact:

- **Simon Martinez R** – `simon.martinez@uantof.cl`
- **Stefan Escaida** – `stefan.escaida@uoh.cl`

During LACORO 2025, technical support sessions and Q&A slots will be scheduled to assist teams with installation, setup, and clarification of rules.